CyberSentinel: Big Data-Based Network Intrusion Detection Using Hadoop HDFS, PySpark and Random Forest

Submitted by

Inchara M P (2023BCS0129)

Guided by

Dr. John Paul Martin

Indian Institute of Information Technology Kottayam

1. Introduction

Intrusion detection systems (IDSs) monitor network traffic and raise alerts for activity such as denial of service, brute-force logins, botnet communication or web attacks. Machine-learning IDSs are usually trained on network-flow records, where each flow is described by numeric statistics such as duration, packet counts, packet lengths and inter-arrival times [1, 2]. Public flow datasets are large: the CSE-CIC-IDS2018 release used in this project contains more than sixteen million flows in ten CSV files [3, 2], which makes storage, cleaning and model training a data-engineering problem as well as a modelling one.

This project, **CyberSentinel**, builds a batch Big Data pipeline for that task. The processed CSE-CIC-IDS2018 files are stored in Hadoop HDFS [4], cleaned and converted into feature vectors with PySpark [5], and classified as Benign or Attack by a Random Forest from Spark MLlib [6, 7]. A Streamlit dashboard presents the results and runs predictions on uploaded CSV files using the model stored in HDFS. Everything runs on a single Ubuntu virtual machine with 8 GB of RAM, so the work is a Big Data-oriented prototype rather than a multi-node deployment.

A high test accuracy on a random split of a public dataset says little about how a detector behaves on attacks it has not been trained on [8, 9]. Besides the standard evaluation, the project therefore reports three further analyses:

- a file-by-file **data-quality audit** of the ten CSV files, recording schema differences, repeated header rows and invalid values behind each cleaning decision;
- an **attack-family breakdown** of the missed attacks, showing which family accounts for the false negatives of the main model;
- a **known-versus-unseen evaluation**, in which each attack family is withheld from training in turn and the Random Forest is compared with a K-Means anomaly detector trained only on benign traffic and with a hybrid of the two, at false-alarm rates calibrated on separate benign data.

A repeated timing benchmark is also reported to show the run-to-run variability of prediction throughput. Section 2 reviews related work, Section 3 states the problem and research questions, Section 4 describes the architecture and methods, Section 5 presents and analyses the results, and Section 6 concludes. The source code and a step-by-step reproduction guide are available in the project repository [10].

2. Literature Survey

2.1. Intrusion-detection datasets and their quality

Sharafaldin, Lashkari and Ghorbani described how the CIC datasets are generated: benign background traffic is produced from user profiles, attacks are run from a separate network, and flows are extracted with CICFlowMeter into more than 80 features [1]. CSE-CIC-IDS2018 follows the same approach on a larger AWS testbed and is distributed as packet captures and processed CSV files, one per capture day [3]. Ring et al. surveyed network-based IDS datasets and pointed out that properties such as labelling, recording period and class balance strongly affect what a model trained on them can show [11].

Several studies have examined the quality of the CIC datasets directly. Engelen, Rimmer and Joosen found labelling and flow-construction errors in CICIDS2017 [12]. Liu et al. extended this analysis to CIC-IDS-2017 and CSE-CIC-IDS-2018 and reported problems in attack orchestration, feature generation and labelling, and released corrected versions of both datasets [13]. Leevy and Khoshgoftaar's survey of CSE-CIC-IDS2018 studies noted that many papers do not describe their data cleaning in enough detail to be reproduced [2]. These works motivate the audit carried out in this project, which records what is removed or replaced and why.

2.2. Machine learning on CSE-CIC-IDS2018

Leevy and Khoshgoftaar reviewed intrusion-detection models built on CSE-CIC-IDS2018 and observed that very high reported scores may reflect overfitting, that class imbalance is often not addressed, and that results are hard to compare across papers [2]. Karataş, Demir and Şahingöz applied oversampling to the minority attack classes and compared several classifiers, reporting results per attack class [14]. Songma, Sathuphan and Pamutha combined data cleaning, normalisation and principal component analysis with several classifiers and reported strong results for XGBoost, decision trees and Random Forest [15]. Random Forest itself, introduced by Breiman, is an ensemble of randomised decision trees whose votes are combined, and it provides feature-importance estimates as a by-product [6].

2.3. Big Data platforms for intrusion detection

HDFS stores large files as replicated blocks managed by a NameNode and is designed for high-throughput sequential access [4, 16]. Apache Spark keeps intermediate data in memory across a cluster and provides a DataFrame API [5], on which MLlib offers scalable implementations of classifiers such as Random Forest together with feature transformers and evaluators [7, 17]. Othman et al. proposed Spark-Chi-SVM, which combines chi-squared feature selection with an SVM in Spark and evaluated it on the KDD99 dataset [18]. Atefinia and Ahmadi compared six Spark MLlib algorithms on

the botnet portion of CSE-CIC-IDS2018 on clusters of one to eight workers, and reported that more workers reduced training time while detection scores were essentially unchanged [19]. Mamdouh et al. built a Spark pipeline on CSE-CIC-IDS2018 that ranks features by Random Forest importance and classifies traffic with a weighted Random Forest, reporting a weighted F1-score of about 0.999 with 34 features [20]. Spark-based Random Forest detection on this dataset is therefore not new, and this project does not claim it as a contribution.

2.4. Generalisation and anomaly detection

Sommer and Paxson argued that machine learning in intrusion detection is much better at finding things similar to what it has already seen than at finding genuinely new attacks, and that evaluations should reflect this [8]. Arp et al. listed common pitfalls in security machine learning, including sampling bias and data snooping, which can produce scores that do not carry over to practice [9]. Cantone, Marrocco and Bria tested four classifiers on four IDS datasets, including a corrected CSE-CIC-IDS2018: models were almost perfect when trained and tested on the same dataset but mostly no better than chance across datasets [21].

Anomaly detection takes the opposite approach: a model of normal behaviour is learned and large deviations are flagged, which in principle allows detection of attacks not seen before [22]. Kitsune, for example, uses an ensemble of autoencoders trained on benign traffic to detect network attacks online [23]. Clustering-based detectors are a simpler alternative; Spark MLlib provides K-Means with the scalable k-means|| initialisation [24, 7]. Table 1 summarises the works closest to this project.

Table 1: Summary of the most closely related work.

Study	Platform / method	Data	Relation to this work
Leevy & Khoshgoftaar [2]	Survey	CSE-CIC-IDS2018	Overfitting, imbalance and reproducibility concerns
Liu et al. [13]	Dataset error analysis	CIC-IDS-2017/2018	Documents data errors; motivates the audit
Karataş et al. [14]	Oversampling + ML	CSE-CIC-IDS2018	Per-class results on the same dataset
Atefinia & Ahmadi [19]	Six Spark MLlib models	Botnet subset	Spark on CSE-CIC-IDS2018, multi-worker timing
Mamdouh et al. [20]	Spark + weighted RF	CSE-CIC-IDS2018	Closest prior work: same platform and model
Cantone et al. [21]	Four classifiers	Four datasets	Cross-dataset failure of high in-dataset scores
This project	HDFS + Spark RF, K-Means, hybrid	Nine files, 8.28 M flows	Audit, missed-attack analysis, unseen families

Together, this literature shows that Spark-based Random Forest detection, class imbalance and generalisation have each been studied on this dataset. The question addressed here is narrower: within one reproducible, resource-limited Spark pipeline,

how do the dataset's quality issues, the distribution of missed attacks across families and the detection of withheld families affect what the headline accuracy actually means? To our knowledge this combination has not been reported for CSE-CIC-IDS2018, although a more exhaustive search would be needed to confirm it.

3. Problem Statement

Network-flow datasets such as CSE-CIC-IDS2018 are too large to process comfortably with single-process tools, and their CSV files contain inconsistent headers, repeated header rows and non-finite numeric values. Model scores reported on them are often hard to interpret because cleaning steps, class balance and per-class errors are not documented [2, 13]. In addition, a random train/test split draws the test attacks from the same families as the training attacks, so it measures recognition of known attacks only [8]. The problem addressed is to build a traceable HDFS and PySpark pipeline that cleans and classifies this data, and to evaluate it in a way that shows where the classifier works and where it fails. The work is organised around four research questions:

RQ1. How can schema differences, repeated headers and non-finite values in the processed files be detected, handled and documented reproducibly?

RQ2. How well does a Spark Random Forest separate benign from attack traffic on a held-out test set, and which attack families account for the missed attacks?

RQ3. How well does the Random Forest detect an attack family withheld from training, and can a benign-only anomaly detector, alone or combined with the Random Forest, recover such attacks at an acceptable false-alarm rate?

RQ4. How do prediction time and throughput vary with batch size on a single-machine Spark set-up, and how repeatable are these timings?

The scope is offline, binary classification on one virtual machine. Packet capture, streaming, multi-class labelling and production deployment are outside the scope, and withholding a family from a public benchmark simulates a new attack type but is not a real zero-day test.

4. Proposed Solution / Architecture

4.1. System architecture

Figure 1 shows the pipeline. The processed CSV files are copied to HDFS, read and cleaned by PySpark, assembled into a 78-dimensional feature vector and used to train the Random Forest. The cleaned data are written back to HDFS in Parquet format, and every later stage (training, evaluation, cross-validation, benchmarking) reads this Parquet copy instead of re-parsing the CSV files. The trained model is saved both locally and in

HDFS, from where the Streamlit dashboard loads it for CSV-based prediction. Metrics and figures are written to a results folder that the dashboard and this report use.

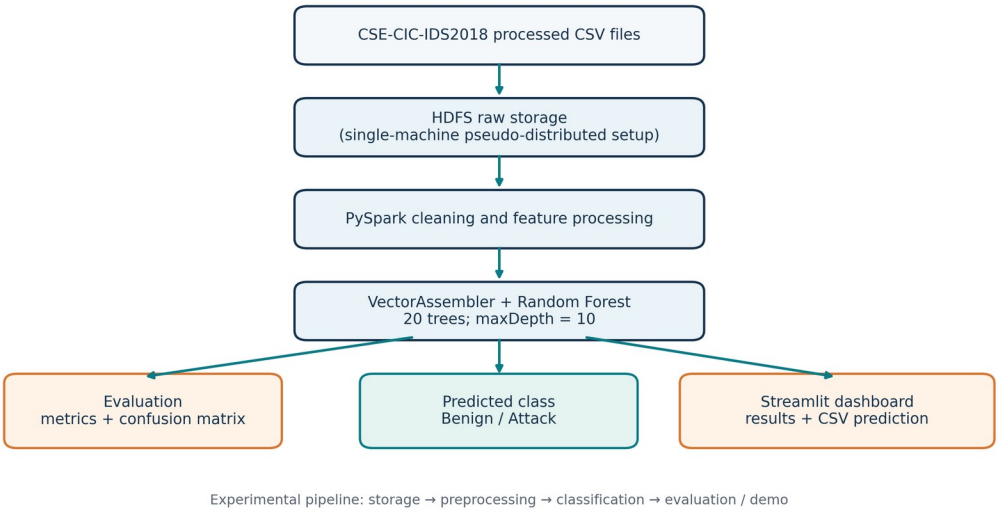


Figure 1: CyberSentinel pipeline: HDFS storage, PySpark preprocessing, Random Forest classification, and evaluation and dashboard outputs.

4.2. Dataset and preprocessing

Nine of the ten processed CSE-CIC-IDS2018 files share an 80-column header and were used for modelling. The Tuesday 20-02-2018 file was excluded because its header has 84 columns (Section 5.1). Each record has a timestamp, the original text label and 78 numeric flow features. Preprocessing removes rows whose `Label` value is the literal header text, maps Benign to 0 and every other label to 1, replaces missing, NaN and ±Infinity values in the 78 features with zero, and assembles the features with `VectorAssembler`. The data are then split 80/20 at random with seed 42. Table 2 lists the resulting counts.

Table 2: Record counts through preprocessing.

Stage	Records	Note
Rows read from the nine files	8,284,254	Spark CSV read
Repeated header rows removed	59	Label = "Label"
Cleaned working dataset	8,284,195	6,112,151 benign (73.8%), 2,172,044 attack (26.2%)
Training partition	6,628,175	80%, seed 42
Held-out test partition	1,656,020	20%, seed 42

4.3. Data-quality audit

A separate script, `data_quality_audit.py`, reads each of the ten files on its own, with its own header and with every value kept as text so that literal `NaN` and `Infinity` strings are counted exactly. For each file it records the column count and any columns

that differ from the shared schema, the rows read, repeated header rows, missing, NaN and Infinity values per feature, the labels present, and whether the file is included. The outputs are saved as CSV and JSON so that each cleaning decision rests on recorded evidence.

4.4. Random Forest model and evaluation

The classifier is Spark MLlib's `RandomForestClassifier` with 20 trees, maximum depth 10 and seed 42, a configuration chosen to fit the 8 GB machine rather than through a parameter search. It is trained on the training partition and evaluated once on the held-out test partition. Overall accuracy and weighted precision, recall and F1-score are computed with Spark's evaluator; attack precision, attack recall, attack F1-score and false-negative rate (FN / (FN + TP)) are computed from the confusion matrix because a high overall score can hide missed attacks. Training accuracy is also measured, and a three-fold `CrossValidator` is run on a 500,135-record sample of the training partition to check the stability of the result.

4.5. Known versus unseen attack families

The 13 attack labels in the nine files were grouped into six families (Table 3). The text label is kept only for this grouping and is never a model input. The cleaned data are split once with seed 42 into a 70% training pool, a 10% calibration set, of which only the benign rows are used, and a 20% test set. This test set contains exactly the same 1,220,899 benign and 435,121 attack records as the 20% test split of Section 4.2, because Spark draws the same per-row random values for the same data and seed.

Table 3: Attack families and their record counts in the test set *T* (main evaluation) and in the training pool plus test set (robustness check).

Family	Labels in the dataset	Test (T)	Pool + test
Bot	Bot	57,522	257,577
Brute-force	FTP-BruteForce, SSH-Bruteforce	76,112	342,873
DDoS	DDOS attack-HOIC, DDOS attack-LOIC-UDP	137,982	619,073
DoS	DoS attacks-Hulk, -SlowHTTPTest, -GoldenEye, -Slowloris	130,705	588,688
Infiltration	Infilteration	32,611	145,744
Web	Brute Force -Web, Brute Force -XSS, SQL Injection	189	831

For every family, a Random Forest with the settings of Section 4.4 is trained without any record of that family. A reference forest is trained on all families. All four detectors are then compared on **the same records**: the family's records in the test set *T*. The reference forest gives the “seen” recall, the forest trained without the family gives the “unseen” recall, and the anomaly detector and the hybrid are scored on the same rows. Because none of the family's pool records were used to train the forest without that family, its unseen recall is also measured on all of the family's records in $P \cup T$, a larger

sample, as a robustness check. Exact detected counts are recorded for every detector. To fit in memory, each forest is trained on a random sample of at most 1,000,000 rows from the pool; evaluation always uses the full data. The anomaly detector standardises the features and fits K-Means with k = 10 on 200,000 benign training rows; a flow's anomaly score is its squared distance to the nearest cluster centre. Two thresholds are taken as the 99th and 99.9th percentiles of the score on the 611,462 benign calibration rows, giving target false-alarm rates of 1% and 0.1%. They are never tuned on test data. The hybrid flags a flow if either the Random Forest or the anomaly detector flags it. Algorithm 1 summarises the procedure.

Algorithm 1 Known-versus-unseen evaluation of the Random Forest (RF), anomaly detector (AD) and hybrid

Split cleaned data (seed 42) into pool P (70%), calibration C (10%), test T (20%)
$s \leftarrow$ StandardScaler fitted on benign rows of P; $K \leftarrow$ K-Means(k = 10) on 200,000 scaled benign rows
score(x) $\leftarrow$ min over centres c of $\|s(x) - c\|^2$
$\tau_1 \leftarrow$ 99th percentile, $\tau_2 \leftarrow$ 99.9th percentile of score on benign rows of C
$RF_{all} \leftarrow$ RandomForest(sample $\leq$ 1 M rows of P); record seen recall per family on T
for each family f in {Bot, Brute-force, DDoS, DoS, Infiltration, Web} **do**
    RF–f $\leftarrow$ RandomForest(sample $\leq$ 1 M rows of P with family f removed)
    $E \leftarrow$ records of f in T (main result); repeat with $P \cup T$ (robustness check)
    for τ in $\{\tau_1, \tau_2\}$ **do**
        AD(x) $\leftarrow$ score(x) > τ; Hybrid(x) $\leftarrow$ RF–f(x) **or** AD(x)
        record recall of RF–f, AD and Hybrid on E
    end for
end for
Report false-alarm rates of RF_{all}, AD and Hybrid on the benign rows of T

4.6. Prediction benchmark

A one-million-record subset of the cleaned feature data is persisted to disk, and a 10,000-record warm-up prediction is run first. For 100,000, 250,000, 500,000 and 1,000,000 records, the saved model predicts the batch, the predictions are grouped and counted by class, and the counts are collected; wall-clock time and records per second are recorded. The original benchmark measured each size once. The repeated benchmark runs each size three times in rotation, so that a slow-down of the machine affects all sizes, and reports the mean and standard deviation.

4.7. Implementation

The system runs on Ubuntu 24.04 in VirtualBox (8 GB RAM, 4 vCPUs) with Java 17, Hadoop 3.3.6 in single-node mode (replication 1), Python 3.12 and PySpark 4.2 in local mode with 3 GB of driver memory. The raw files are stored under `/cybersecurity/CSE_CIC_IDS2018/raw/` in HDFS and the model under `.../results/random_forest_model`. A shared module (`common.py`) holds the file

list, Spark session and cleaning code, and the stages are run in order: `data_preprocessing.py`, `dataset_statistics.py`, `data_quality_audit.py`, `train_model.py`, `evaluate_model.py`, `cross_validation.py`, `scalability_experiment.py`, `generate_plots.py` and `unseen_attack_experiment.py`. Because HDFS is the default file system of the Spark session, local paths are passed to Spark as explicit `file://` URIs.

The Streamlit dashboard (`app.py`) has six pages: Dashboard Overview, Model Evaluation, Performance Analysis, Data Quality Audit, Known vs Unseen Attacks, and CSV Anomaly Detection. The last page accepts a CSV with the 78 feature columns, aligns them to the training order, replaces invalid values with zero, loads the model from HDFS and shows and exports the predictions. The dataset is not included in the repository; the README and `data/README.md` give the download and set-up commands [10, 3].

5. Results & Analysis

5.1. Data-quality audit (RQ1)

Table 4 summarises the audit of the ten files. Nine files share the same 80-column header. The Tuesday 20-02-2018 file has 84 columns because it contains four extra identifier columns (Flow ID, Src IP, Src Port, Dst IP), which shift every later column. This explains the schema warning seen when all files were read together, where a value such as `Idle Mean` appeared in the `Label` position. The file is schema-incompatible rather than corrupt.

Table 4: Per-file audit of the ten processed CSV files.

Capture day	Cols	Rows read	Rep. headers	NaN	Infinity	Decision
Fri 02-03-2018	80	1,048,575	0	2,558	5,542	Included
Fri 16-02-2018	80	1,048,575	1	0	0	Included
Fri 23-02-2018	80	1,048,575	0	3,754	7,662	Included
Tue 20-02-2018	84	7,948,748	0	36,767	82,139	Excluded
Thu 01-03-2018	80	331,125	25	1,834	4,004	Included
Thu 15-02-2018	80	1,048,575	0	4,921	11,133	Included
Thu 22-02-2018	80	1,048,575	0	3,569	7,651	Included
Wed 14-02-2018	80	1,048,575	0	2,277	5,371	Included
Wed 21-02-2018	80	1,048,575	0	0	0	Included
Wed 28-02-2018	80	613,104	33	4,041	8,297	Included

The excluded file is large: it holds 7,948,748 of the 16,233,002 rows in all ten files (49.0%), including 7,372,557 benign flows and all 576,191 DDoS-LOIC-HTTP records, so that attack type is absent from all experiments here. Repeated header rows occurred in three files (1, 25 and 33 rows), which accounts for the 59 rows removed in Table 2. No feature had blank values. Invalid values were confined to two of the 78 features, both

per-second rates: Flow Byts/s (22,954 NaN and 13,353 Infinity) and Flow Pkts/s (36,307 Infinity). Each has exactly 36,307 invalid entries (0.44% of the working rows), which is consistent with flows of zero recorded duration. These values are replaced with zero, which is simple but may distort those two features.

Two further observations limit the scope of the results. Seven of the nine used files contain exactly 1,048,575 data rows, one fewer than the row limit of a Microsoft Excel worksheet, which suggests they may have been truncated before release; the audit cannot confirm this. Each attack type was also recorded on specific days, for example Infiltration on 28-02 and 01-03 and the web attacks on 22-02 and 23-02, so withholding a family in Section 5.4 also withholds part of the traffic of those days.

5.2. Overall classification results (RQ2)

Table 5 reports the results of the Random Forest on the 1,656,020-record held-out test set, and Figure 2 shows the confusion matrix. Test accuracy was 97.66%, close to the training accuracy of 97.68% and to the three-fold cross-validation accuracy of 97.63%, so there is no accuracy-based sign of severe overfitting on this split. Attack precision (98.55%) is clearly higher than attack recall (92.47%): when the model flags an attack it is almost always right, but 32,779 attack flows (7.53%) were classified as benign, against 5,907 benign flows flagged as attacks.

Table 5: Random Forest results on the held-out test set. *Mean over three folds of a 500,135-record training sample.

Metric	Value	Metric	Value
Test accuracy	97.66%	Attack precision	98.55%
Weighted precision	97.68%	Attack recall	92.47%
Weighted recall	97.66%	Attack F1-score	95.41%
Weighted F1-score	97.64%	False-negative rate	7.53%
Training accuracy	97.68%	3-fold CV accuracy*	97.63%

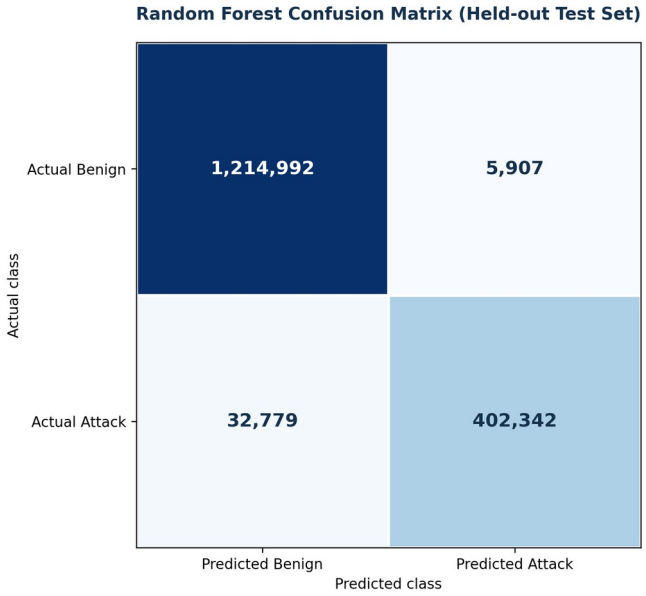


Figure 2: Confusion matrix on the 1,656,020-record held-out test set (TN 1,214,992; FP 5,907; FN 32,779; TP 402,342).

Figure 3 shows the fifteen most important features. Maximum packet length (0.082), destination port (0.071), minimum forward segment size (0.067), maximum flow inter-arrival time (0.055) and forward packets per second (0.049) have the largest importance scores. These describe what the fitted forest relies on in this dataset; they do not imply that the features cause attacks or would rank the same on other data.

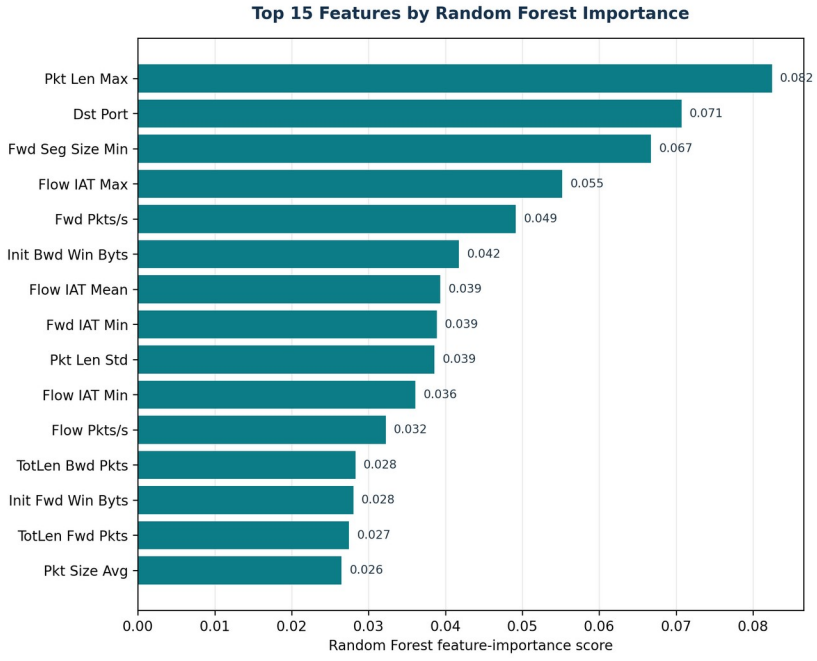


Figure 3: Top 15 Random Forest feature-importance scores.

5.3. Which attacks are missed

Table 6 breaks down the test results of the reference forest of Section 4.5 by family. Bot, Brute-force, DDoS and DoS are detected almost perfectly. Infiltration, although

included in training, is detected only 0.65% of the time and accounts for 32,398 of the 32,814 missed attacks, that is 98.7% of all false negatives. In these 78 flow features, Infiltration traffic is largely indistinguishable from benign traffic for this model. Two points should be noted when comparing this table with Section 5.2. First, the reference forest is not the main model: it is trained on a sample of at most one million rows rather than the full training partition, although it is tested on the same 435,121 attack records, so its total of 32,814 missed attacks is close to, but not the same as, the main model's 32,779 false negatives. Second, the counts in Table 6 are exact counts from the predictions, not estimates. The breakdown therefore strongly suggests, but does not directly measure, that the main model's false negatives are also dominated by Infiltration. Web attacks are also poorly detected, but they have only 189 test records.

Table 6: Detection per family by the reference Random Forest of the unseen-family experiment (all families, trained on ≤ 1 M rows), on the test set. Missed counts are exact.

Family	Test records	Detected	Missed
Bot	57,522	99.86%	81
Brute-force	76,112	99.99%	5
DDoS	137,982	99.90%	138
DoS	130,705	99.98%	24
Infiltration	32,611	0.65%	32,398
Web	189	11.11%	168
Total	**435,121**		**32,814**

5.4. Detection of unseen attack families (RQ3)

Table 7 and Figure 4 compare each family when it is seen in training with the case where it is withheld, with every detector scored on the same test records. Without a family in training, supervised detection collapses for most families: Bot falls from 99.9% to 0.0%, DDoS from 99.9% to 2.6% and DoS from 100.0% to 21.5%. Brute-force keeps 50.7%, which suggests partial overlap with patterns learned from other families. The high overall accuracy therefore applies to attack types represented in training, not to new ones.

Table 7: Detection rate (recall) per family, all columns measured on the same test records (*T*). RF seen: reference forest trained on all families. RF unseen, AD and Hybrid: family withheld from Random Forest training. AD = benign-only K-Means anomaly detector at the 1% or 0.1% calibrated threshold.

Family	Records	RF seen	RF unseen	AD 1%	Hybrid 1%	AD 0.1%	Hybrid 0.1%
Bot	57,522	99.9%	0.0%	0.1%	0.1%	0.0%	0.0%
Brute-force	76,112	100.0%	50.7%	16.7%	50.7%	0.0%	50.7%
DDoS	137,982	99.9%	2.6%	0.3%	2.9%	0.3%	2.9%
DoS	130,705	100.0%	21.5%	1.4%	22.7%	0.4%	21.8%
Infiltration	32,611	0.7%	0.7%	2.6%	3.3%	0.9%	1.6%
Web	189	11.1%	5.8%	40.7%	46.6%	25.4%	31.2%

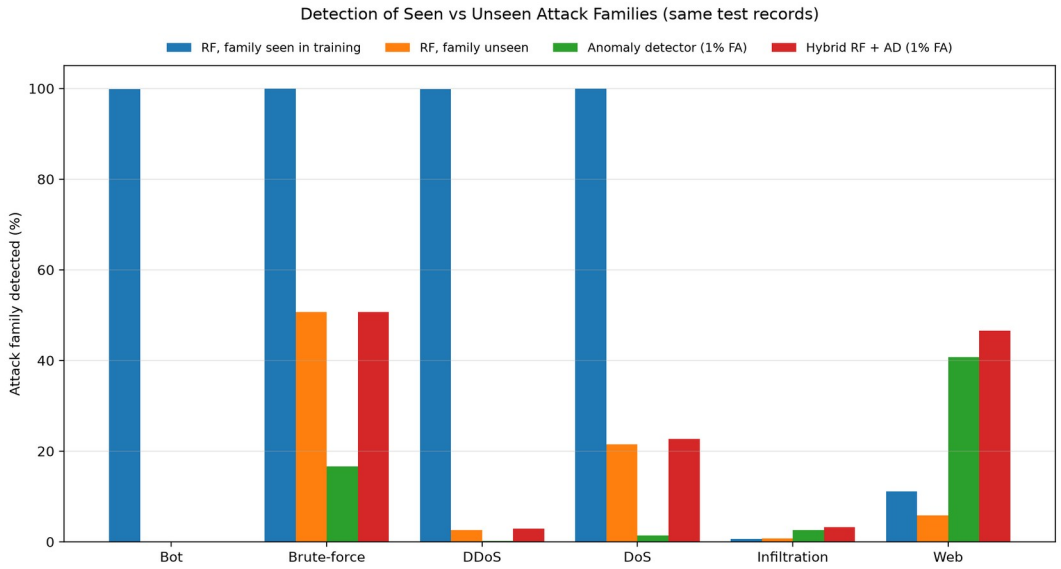


Figure 4: Detection of each attack family on the same test records: seen in training, withheld from training, and by the anomaly detector and the hybrid at the 1% threshold.

The benign-only anomaly detector helps only where attack flows lie far from normal traffic. It detects 40.7% of the unseen web attacks, which raises hybrid detection of that family from 5.8% to 46.6% at the 1% threshold, and it raises Infiltration from 0.7% to 3.3%. For Bot, DDoS and DoS it adds almost nothing (0.08%, 0.26% and 1.43%), so these flows fall within the range of benign variation as seen by a distance-based detector on standardised features. This is a negative result for that detector design, and it is reported as such. Measuring the unseen detectors on all of each family's pool and test records instead (the robustness check) changed every recall in Table 7 by at most 1.2 percentage points, so the conclusions do not depend on which records are used.

Table 8 gives the cost on the 1,220,899 benign test flows. The thresholds, chosen only on benign calibration data, produced test false-alarm rates of 0.99% and 0.11% against targets of 1% and 0.1%, so the calibration transferred to unseen benign traffic. At the 0.1% threshold the hybrid adds about 1,100 false alarms per million benign flows

over the Random Forest alone, while still raising unseen web-attack detection from 5.8% to 31.2%; at the 1% threshold the extra cost is about 9,900 per million. The forests trained without one family had benign false-alarm rates between 0.10% and 0.59%.

Table 8: False-alarm cost on benign test traffic.

Detector	False-alarm rate	Per million benign flows
Random Forest (all families)	0.477%	4,770
Anomaly detector, 1% threshold	0.990%	9,900
Hybrid, 1% threshold	1.467%	14,670
Anomaly detector, 0.1% threshold	0.110%	1,100
Hybrid, 0.1% threshold	0.587%	5,870

5.5. Prediction benchmark (RQ4)

Table 9 and Figure 5 compare the repeated benchmark with the original single run. The repeated runs were slower and much more variable: one million records took 11.48 ± 1.36 s on average (about 88,000 records/s) against 8.84 s in the single run, and at 500,000 records the standard deviation (5.93 s) exceeded half the mean (9.99 s). Throughput still tended to rise with batch size, from about 46,000 to 88,000 records per second, consistent with fixed per-job overheads being spread over more records, but the 250,000 and 500,000 means overlap within one standard deviation. The single-run figures lie at the favourable end of what this VM delivers, and on a shared desktop VM performance is better reported as a range from repeated runs. All timings are for one machine and do not measure multi-node scaling.

Table 9: Prediction benchmark: mean ± standard deviation of three runs, compared with the original single run.

Records	Time, 3 runs (s)	Single run (s)	Throughput, 3 runs (rec/s)	Single run (rec/s)
100,000	2.26 ± 0.57	1.36	46,117 ± 11,475	73,787
250,000	4.48 ± 2.19	3.24	63,740 ± 24,312	77,203
500,000	9.99 ± 5.93	4.74	60,837 ± 27,546	105,564
1,000,000	11.48 ± 1.36	8.84	87,961 ± 10,657	113,096

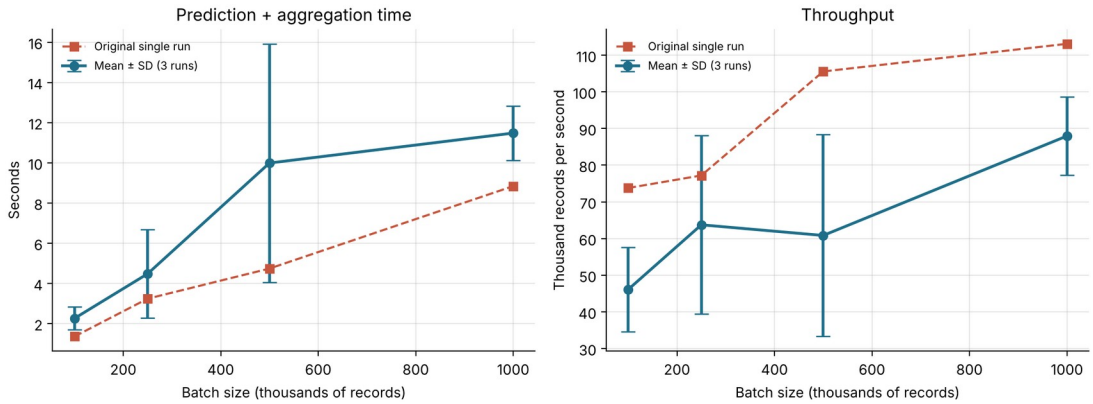


Figure 5: Repeated benchmark (mean ± SD over three runs per size) compared with the original single run.

5.6. Dashboard check and discussion

As a functional test of the dashboard, a synthetic 200-row CSV with the 78 required columns was uploaded; the app loaded the model from HDFS and returned 197 Benign and 3 Attack predictions. Since these rows have no true labels, this checks the upload-to-prediction path only, not accuracy.

Taken together, the results change how the 97.66% accuracy should be read. It describes attack types that were present in training; in the reference forest, nearly all of the missed attacks belong to one family, Infiltration; and a family absent from training is mostly missed. A benign-only anomaly detector is not a general remedy: it helps for attacks whose flows look unusual, such as the web family, at a measurable false-alarm cost, and does little for flood-style or bot traffic. An operational detector would therefore need regular retraining on newly labelled attacks as well as anomaly detectors chosen for the behaviour they are meant to catch, with an explicit false-alarm budget.

The results have several limits. Everything ran on a single VM, so the Big Data components are exercised but horizontal scaling is not measured. Half of the dataset's rows, including all DDoS-LOIC-HTTP traffic, were excluded with the Tuesday file, and seven files may be truncated. All attacks were merged into one class for the main model, invalid values were set to zero, and cross-validation used one parameter setting on a sample. The unseen-family forests were trained on samples of at most one million rows, the anomaly detector is a simple K-Means model, the web family has only 189 test records (831 in pool and test), and withholding a family also withholds day-specific traffic. Finally, the original CSE-CIC-IDS2018 files contain the labelling errors reported by Liu et al. and Cantone et al. [13, 21], and all results here are subject to them.

6. Conclusion

CyberSentinel implements a batch Big Data pipeline for network intrusion detection, combining HDFS storage, PySpark preprocessing, a Spark MLlib Random Forest and a

Streamlit dashboard, and applies it to 8,284,195 cleaned flows from nine CSE-CIC-IDS2018 files. On the held-out test set the model reached 97.66% accuracy, 98.55% attack precision and 92.47% attack recall, with training and cross-validation accuracy within 0.05 percentage points of the test result.

The additional analyses show what this score does and does not mean. The file audit documented an excluded 84-column file holding 49% of all rows, 59 repeated headers and 36,307 invalid values in each of two rate features. In the reference forest of the unseen-family experiment, about 99% of the missed attacks are Infiltration, which it detects only 0.65% of the time even when trained on it. When a family is withheld from training, detection collapses for most families (Bot 0.0%, DDoS 2.6%). A benign-only K-Means detector recovers part of the unseen web attacks (hybrid 46.6% at a 1% threshold, 31.2% at 0.1%, against 5.8% for the Random Forest alone, all on the same test records) at a cost of about 9,900 or 1,100 extra false alarms per million benign flows, but adds little for flood-style and bot traffic. Repeated timing runs showed that single-run throughput figures were optimistic on the shared VM.

Future work includes aligning the excluded file by dropping its four identifier columns and adding its DDoS-LOIC-HTTP traffic; repeating the experiments on the corrected dataset releases [13, 21]; investigating features or models that separate Infiltration from benign traffic; evaluating stronger anomaly detectors such as autoencoders [23] on the withheld families; adding a day-based holdout to separate attack-type effects from day-to-day changes; comparing Random Forest with other classifiers under identical splits; and running the benchmark on an idle machine and on a multi-node Spark cluster.

References

[1] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), 2018, pp. 108–116. doi:10.5220/0006639801080116.

[2] J. L. Leevy, T. M. Khoshgoftaar, A survey and analysis of intrusion detection models based on CSE-CIC-IDS2018 Big Data, Journal of Big Data 7 (2020) 104. doi:10.1186/s40537-020-00382-x.

[3] Canadian Institute for Cybersecurity, CSE-CIC-IDS2018 on AWS, University of New Brunswick, dataset description and download instructions. https://www.unb.ca/cic/datasets/ids-2018.html.

[4] K. Shvachko, H. Kuang, S. Radia, R. Chansler, The Hadoop distributed file system, in: 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST), 2010, pp. 1–10. doi:10.1109/MSST.2010.5496972.

[5] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, I. Stoica, Apache

Spark: A unified engine for big data processing, Communications of the ACM 59 (11) (2016) 56–65. doi:10.1145/2934664.

[6] L. Breiman, Random forests, Machine Learning 45 (2001) 5–32. doi:10.1023/A:1010933404324.

[7] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. B. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia, A. Talwalkar, MLlib: Machine learning in Apache Spark, Journal of Machine Learning Research 17 (34) (2016) 1–7. https://jmlr.org/papers/v17/15-237.html.

[8] R. Sommer, V. Paxson, Outside the closed world: On using machine learning for network intrusion detection, in: 2010 IEEE Symposium on Security and Privacy, 2010, pp. 305–316. doi:10.1109/SP.2010.25.

[9] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, K. Rieck, Dos and don'ts of machine learning in computer security, in: 31st USENIX Security Symposium, 2022, pp. 3971–3988. https://www.usenix.org/conference/usenixsecurity22/presentation/arp.

[10] Inchara M P, CyberSentinel source code, README with step-by-step reproduction instructions and data download guide, GitHub repository. https://github.com/idsaturn07/cybersentinel.

[11] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, A. Hotho, A survey of network-based intrusion detection data sets, Computers & Security 86 (2019) 147–167. doi:10.1016/j.cose.2019.06.005.

[12] G. Engelen, V. Rimmer, W. Joosen, Troubleshooting an intrusion detection dataset: the CICIDS2017 case study, in: 2021 IEEE Security and Privacy Workshops (SPW), 2021, pp. 7–12. doi:10.1109/SPW53761.2021.00009.

[13] L. Liu, G. Engelen, T. Lynar, D. Essam, W. Joosen, Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018, in: 2022 IEEE Conference on Communications and Network Security (CNS), 2022. doi:10.1109/CNS56114.2022.9947235.

[14] G. Karataş, Ö. Demir, Ö. K. Şahingöz, Increasing the performance of machine learning-based IDSs on an imbalanced and up-to-date dataset, IEEE Access 8 (2020) 32150–32162. doi:10.1109/ACCESS.2020.2973219.

[15] S. Songma, T. Sathuphan, T. Pamutha, Optimizing intrusion detection systems in three phases on the CSE-CIC-IDS-2018 dataset, Computers 12 (12) (2023) 245. doi:10.3390/computers12120245.

[16] Apache Software Foundation, HDFS users guide – Apache Hadoop 3.3.6 documentation. https://hadoop.apache.org/docs/r3.3.6/hadoop-project-dist/hadoop-hdfs/HdfsUserGuide.html.

[17] Apache Software Foundation, Classification and regression – Spark MLlib documentation. https://spark.apache.org/docs/latest/ml-classification-regression.html.

[18] S. M. Othman, F. M. Ba-Alwi, N. T. Alsohybe, A. Y. Al-Hashida, Intrusion detection model using machine learning algorithm on Big Data environment, Journal of Big Data 5 (2018) 34. doi:10.1186/s40537-018-0145-4.

[19] R. Atefinia, M. Ahmadi, Performance evaluation of Apache Spark MLlib algorithms on an intrusion detection dataset, arXiv preprint, 2022. arXiv:2212.05269.

[20] H. Mamdouh, M. Tarek, A. Radwan, A. Saeed, A. Abdeen, M. Ashraf, K. M. Fouad, I. Abdelbaky, Apache Spark powered: Enhancing network intrusion detection system using random forest, in: 2024 6th Novel Intelligent and Leading Emerging Sciences Conference (NILES), 2024, pp. 289–294. doi:10.1109/NILES63360.2024.10753188.

[21] M. Cantone, C. Marrocco, A. Bria, On the cross-dataset generalization of machine learning for network intrusion detection, IEEE Access 12 (2024). arXiv:2402.10974.

[22] V. Chandola, A. Banerjee, V. Kumar, Anomaly detection: A survey, ACM Computing Surveys 41 (3) (2009) 15. doi:10.1145/1541880.1541882.

[23] Y. Mirsky, T. Doitshman, Y. Elovici, A. Shabtai, Kitsune: An ensemble of autoencoders for online network intrusion detection, in: Network and Distributed System Security Symposium (NDSS), 2018. doi:10.14722/ndss.2018.23204.

[24] B. Bahmani, B. Moseley, A. Vattani, R. Kumar, S. Vassilvitskii, Scalable k-means++, Proceedings of the VLDB Endowment 5 (7) (2012) 622–633. doi:10.14778/2180012.2180017.